

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

JANANI R

(1WA24CS133)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026**

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by JANANI R (**1WA24CS133**), who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - (**23CS4PCADA**) work prescribed for the said degree.

Manjula S
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph. Leet Code	4-9
2	Implement Johnson Trotter algorithm to generate permutations.	10-13
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	14-17
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken. And Leet Code	18-22
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	23-26
6	Implement 0/1 Knapsack problem using dynamic programming.	27-29
7	Implement All Pair Shortest paths problem using Floyd's algorithm. And Leet code	30-34
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	35-39
9	Implement Fractional Knapsack using Greedy technique.	40-42
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	43-45
11	Implement "N-Queens Problem" using Backtracking.	46-48

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1:

Write program to obtain the Topological ordering of vertices in a given digraph.

```
#include<stdio.h>

#include<time.h>

#define max 10

int n,m,a[max][max],q[max],topo[max];

void topologysort(int n,int a[10][10])
{
    int id[30],i,j,front=0,rear=-1,v,k=0,c=0;
    for(i=0;i<n;i++)
    {
        id[i]=0;
    }
    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
        {
            if(a[i][j]==1)
                id[j]+=1;
        }
    }
    for(i=0;i<n;i++)
    {
        if(id[i]==0)
        {
            rear++;
            q[rear]=i;
        }
    }
```

```

    }
while(front<=rear)
{
    v=q[front++];
    topo[k++]=v;
    c++;
    for(j=0;j<n;j++)
    {
        if(a[v][j]==1)
        {
            a[v][j]=0;
            id[j]--;
            if(id[j]==0)
                q[++rear]=j;
        }
    }
}
if(c!=n)
{
    printf("cycle exist, topo order not possible");
    return;
}
printf("topo order is:");
for(i=0;i<n;i++)
{
    printf("%d ",topo[i]);
}
}

int main()

```

```

{
    int i,j;

    clock_t start,end;

    double cpu_time_used;

    printf("enter the no.of vertices:");

    scanf("%d",&n);

    printf("enter the weights of adjacent graph:");

    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
        {
            scanf("%d",&a[i][j]);
        }
    }

    start=clock();

    topologysort(n,a);

    end=clock();

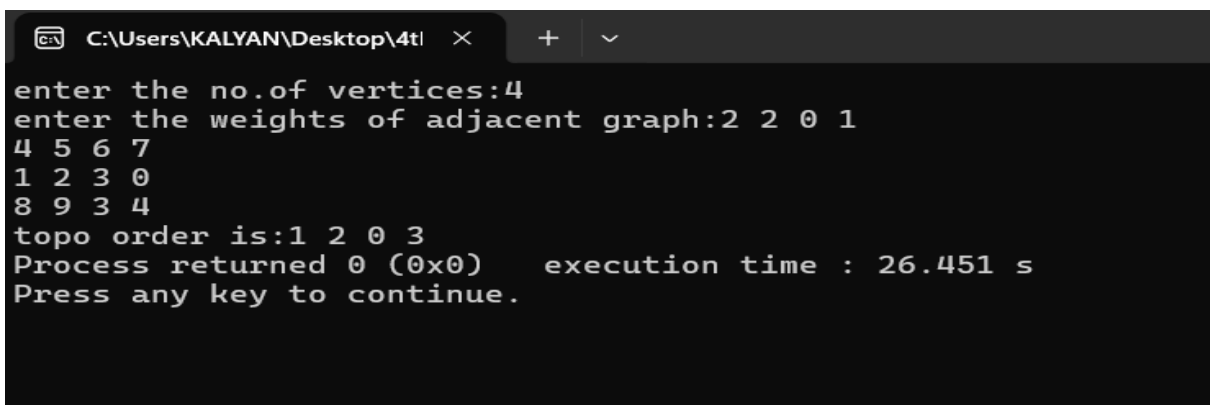
    cpu_time_used=((double)(end-start))/CLOCKS_PER_SEC;

    printf("execution time=%f seconds",cpu_time_used);

    return 0;
}

```

Output:



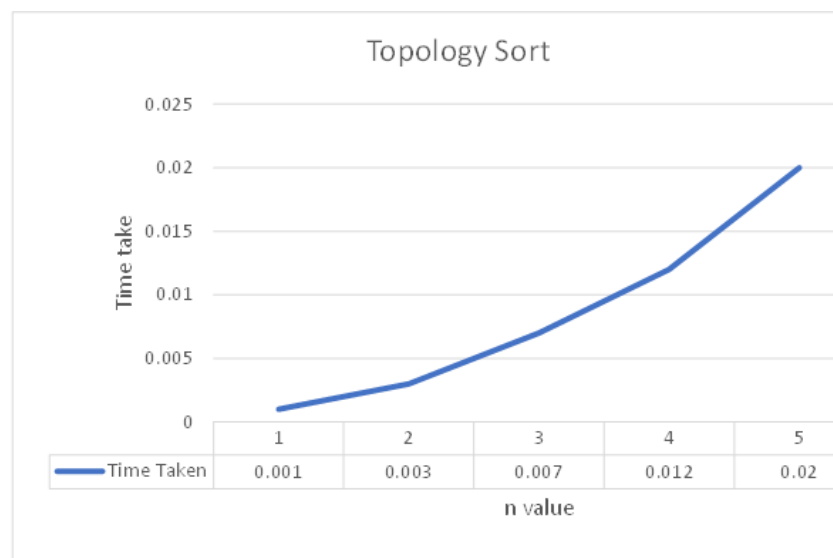
```

C:\Users\KALYAN\Desktop\4tl >
enter the no.of vertices:4
enter the weights of adjacent graph:2 2 0 1
4 5 6 7
1 2 3 0
8 9 3 4
topo order is:1 2 0 3
Process returned 0 (0x0)    execution time : 26.451 s
Press any key to continue.

```

```
C:\Users\KALYAN\Desktop\4tl × + v
enter the no.of vertices:3
enter the weights of adjacent graph:0 1 0
0 0 1
1 0 0
cycle exist, topo order not possible
Process returned 0 (0x0)    execution time : 16.483 s
Press any key to continue.
```

Vertices (n)	Time Taken
10	0.001
20	0.003
30	0.007
40	0.012
50	0.02



Leet Code 1:

Given an array `nums` containing `n` distinct numbers in the range `[0, n]`, return the only number in the range that is missing from the array.

 Code

C   Auto

```
1 int missingNumber(int nums[], int n)
2 {
3     int sum = 0;
4     for(int i = 0; i < n; i++)
5     {
6         sum += nums[i];
7     }
8     int total = (n * (n + 1)) / 2;
9
10    return total - sum;
11 }
```

 Testcase |  Test Result

Accepted Runtime: 0 ms

 Case 1  Case 2  Case 3

Input

nums =
[0,1]

Output

2

Expected

2

☒ Testcase | [>_ Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2 ☒ Case 3

Input

```
nums =  
[3,0,1]
```

Output

```
2
```

Expected

```
2
```

☒ Testcase | [>_ Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2 ☒ Case 3

Input

```
nums =  
[9,6,4,2,3,5,7,0,1]
```

Output

```
8
```

Expected

```
8
```

Lab Program 2:

Implement Johnson Trotter algorithm to generate permutations.

```
#include<stdio.h>

#define left -1
#define right 1

void printpermutation(int perm[],int n)
{
    for(int i=0;i<n;i++)
    {
        printf("%d",perm[i]);
    }
    printf("\n");
}

int getLargeMobile(int perm[],int dir[],int n)
{
    int mobile=0;
    for(int i=0;i<n;i++)
    {
        if(dir[perm[i]-1]==left && i!=0)
        {
            if(perm[i]>perm[i-1]&& perm[i]>mobile)
                mobile=perm[i];
        }
        if(dir[perm[i]-1]==right && i!=n-1)
        {
            if(perm[i]>perm[i+1]&& perm[i]>mobile)
                mobile=perm[i];
        }
    }
}
```

```

    }
    return mobile;
}

void johnsontrotter(int n)
{
    int perm[n],dir[n];
    for(int i=0;i<n;i++)
    {
        perm[i]=i+1;
        dir[i]=left;
    }
    printpermutation(perm,n);
    while(1)
    {
        int mobile=getLargeMobile(perm,dir,n);
        if(mobile==0)
            break;
        int pos;
        for(int i=0;i<n;i++)
        {
            if(perm[i]==mobile)
            {
                pos=i;
                break;
            }
        }
        if(dir[mobile-1]==left)
        {
            int t=perm[pos];

```

```

        perm[pos]=perm[pos-1];
        perm[pos-1]=t;
        pos=pos-1;
    }
    else{
        int t=perm[pos];
        perm[pos]=perm[pos+1];
        perm[pos+1]=t;
        pos=pos+1;
    }
    for(int i=0;i<n;i++)
    {
        if(perm[i]>mobile)
        {
            dir[perm[i]-1]*=-1;
        }
    }
    printpermutation(perm,n);
}
}

int main()
{
    int n;
    printf("enter no.of elements:");
    scanf("%d",&n);
    johnsontrotter(n);
    return 0;
}

```

Output:

```
"C:\Users\KALYAN\Desktop\4" X + v
enter no.of elements:4
1234
1243
1423
4123
4132
1432
1342
1324
3124
3142
3412
4312
4321
3421
3241
3214
2314
2341
2431
4231
4213
2413
2143
2134

Process returned 0 (0x0) execution time : 2.080 s
Press any key to continue.
```

Lab Program 3:

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

```
#include <stdio.h>

#include <time.h>

void mergesort(int a[], int low, int high);

void merge(int a[], int low, int mid, int high);

int main()
{
    clock_t start, end;

    float cpu_time_used;

    int n, i;

    printf("Enter number of elements: ");

    scanf("%d", &n);

    int a[n];

    printf("Enter array elements:\n");

    for(i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }

    start = clock();

    mergesort(a, 0, n - 1);

    end = clock();

    cpu_time_used = ((float)(end - start)) / CLOCKS_PER_SEC;

    printf("\nSorted array:\n");

    for(i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }

    printf("\n\nExecution time = %f seconds", cpu_time_used);
```

```

    return 0;
}

void mergesort(int a[], int low, int high)
{
    int mid;
    if(low < high)
    {
        mid = (low + high) / 2;
        mergesort(a, low, mid);
        mergesort(a, mid + 1, high);
        merge(a, low, mid, high);
    }
}

void merge(int a[], int low, int mid, int high)
{
    int i = low;
    int j = mid + 1;
    int k = low;
    int temp[high + 1];
    while(i <= mid && j <= high)
    {
        if(a[i] <= a[j])
        {
            temp[k++] = a[i++];
        }
        else
        {
            temp[k++] = a[j++];
        }
    }
}

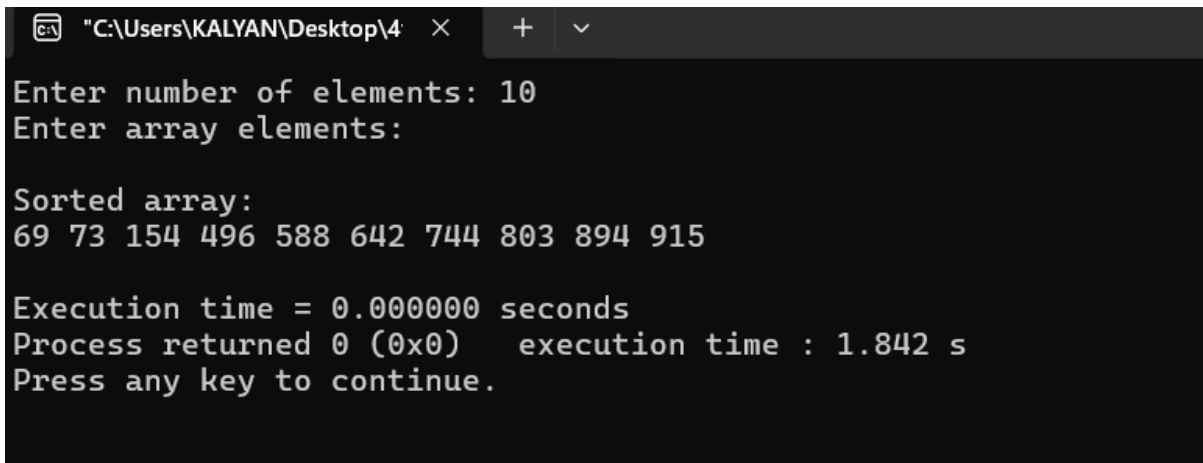
```

```

    }
    while(i <= mid)
    {
        temp[k++] = a[i++];
    }
    while(j <= high)
    {
        temp[k++] = a[j++];
    }
    for(i = low; i <= high; i++)
    {
        a[i] = temp[i];
    }
}

```

Output:



```

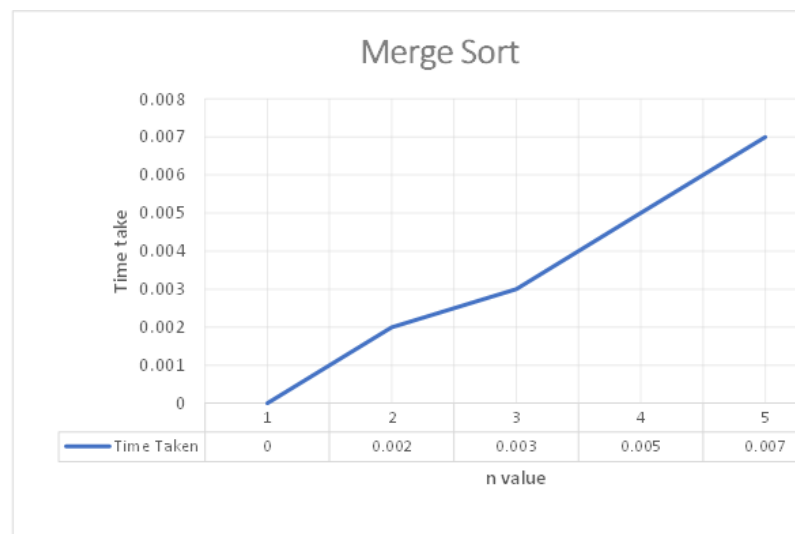
C:\Users\KALYAN\Desktop\4 >
Enter number of elements: 10
Enter array elements:

Sorted array:
69 73 154 496 588 642 744 803 894 915

Execution time = 0.000000 seconds
Process returned 0 (0x0)   execution time : 1.842 s
Press any key to continue.

```


Vertices (n)	Time Taken
1000	0
10000	0.002
20000	0.003
30000	0.005
40000	0.007



Lab Program 4:

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

```
#include <stdio.h>

#include <time.h>

void quicksort(int a[], int low, int high);
int partition(int a[], int low, int high);
int main()
{
    clock_t start, end;
    float cpu_time_used;
    int n, i;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int a[n];
    printf("Enter array elements:\n");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }
    start = clock();
    quicksort(a, 0, n - 1);
    end = clock();
    cpu_time_used =
    ((float)(end - start)) / CLOCKS_PER_SEC;
    printf("\nSorted array:\n");
    for(i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }
}
```

```

printf("\n\nExecution time = %f seconds", cpu_time_used)

return 0;
}

void quicksort(int a[], int low, int high)
{
    int p;
    if(low < high)
    {
        p = partition(a, low, high);
        quicksort(a, low, p - 1);
        quicksort(a, p + 1, high);
    }
}

int partition(int a[], int low, int high)
{
    int pivot, i, j, temp;
    pivot = a[low];
    i = low + 1;
    j = high;
    while(i <= j)
    {
        while(i <= high && a[i] <= pivot)
        {
            i++;
        }
        while(a[j] > pivot)
        {
            j--;
        }
    }

```

```

    if(i < j)
    {
        temp = a[i];
        a[i] = a[j];
        a[j] = temp;
    }
    temp = a[low];
    a[low] = a[j];
    a[j] = temp;
    return j;
}

```

Output:

```

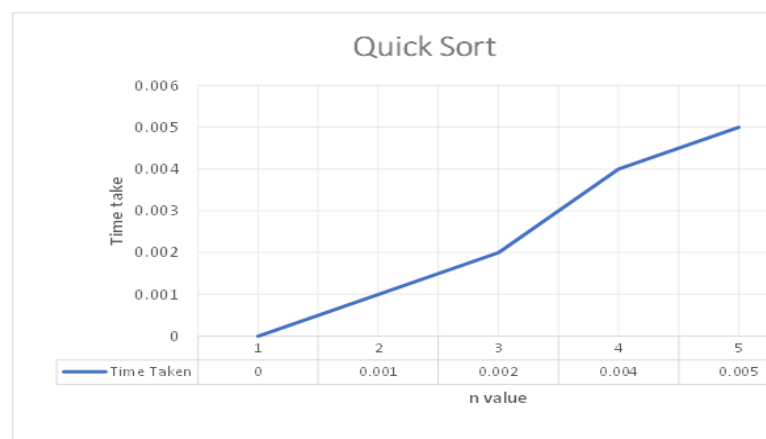
C:\Users\KALYAN\Desktop\4  X  +  v
Enter number of elements: 10
Enter array elements:
12 346 311 890 32 22 89 224 25 00

Sorted array:
0 12 22 25 32 89 224 311 346 890

Execution time = 0.000000 seconds
Process returned 0 (0x0)   execution time : 22.513 s
Press any key to continue.

```

Vertices (n)	Time Taken
1000	0
5000	0.001
20000	0.002
30000	0.004
40000	0.005



Leet Code Program:

Given an array of integers `nums`, sort the array in increasing order based on the frequency of the values. If multiple values have the same frequency, sort them in decreasing order.

C  Auto

```
1  /**
2   * Note: The returned array must be malloced, assume caller calls free().
3   */
4  int* frequencySort(int* nums, int numsSize, int* returnSize) {
5      int freq[201] = {0};
6      for(int i = 0; i < numsSize; i++)
7      {
8          freq[nums[i] + 100]++;
9      }
10     for(int i = 0; i < numsSize - 1; i++)
11     {
12         for(int j = 0; j < numsSize - 1 - i; j++)
13         {
14             int f1 = freq[nums[j] + 100];
15             int f2 = freq[nums[j + 1] + 100];
16             if(f1 > f2 || (f1 == f2 && nums[j] < nums[j + 1]))
17             {
18                 int temp = nums[j];
19                 nums[j] = nums[j + 1];
20                 nums[j + 1] = temp;
21             }
22         }
23     }
24     *returnSize = numsSize;
25     int* ans = (int*)malloc(numsSize * sizeof(int));
26     for(int i = 0; i < numsSize; i++)
27     {
28         ans[i] = nums[i];
29     }
30     return ans;
31 }
```

Output:

Accepted Runtime: 0 ms

 Case 1

 Case 2

 Case 3

Input

nums =
[1,1,2,2,2,3]

Output

[3,1,1,2,2,2]

Expected

[3,1,1,2,2,2]

Accepted Runtime: 0 ms

✓ Case 1 ✓ Case 2 ✓ Case 3

Input

```
nums =  
[2,3,1,3,2]
```

Output

```
[1,3,3,2,2]
```

Expected

```
[1,3,3,2,2]
```

Accepted Runtime: 0 ms

✓ Case 1 ✓ Case 2 ✓ Case 3

Input

```
nums =  
[-1,1,-6,4,5,-6,1,4,1]
```

Output

```
[5,-1,4,4,-6,-6,1,1,1]
```

Expected

```
[5,-1,4,4,-6,-6,1,1,1]
```

Lab Program 5:

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

```
#include <stdio.h>

#include <time.h>

void heapsort(int a[], int n);
void heapify(int a[], int n, int i);

int main()
{
    clock_t start, end;
    float cpu_time_used;

    int n, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int a[n];

    printf("Enter array elements:\n");

    for(i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }

    start = clock();

    heapsort(a, n);

    end = clock();

    cpu_time_used = ((float)(end - start))

    printf("\nSorted array:\n");

    for(i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }

    printf("\n\nExecution time = %f seconds", cpu_time_used);
```

```

    return 0;
}

void heapsort(int a[], int n)
{
    int i, temp;
    for(i = n / 2 - 1; i >= 0; i--)
    {
        heapify(a, n, i);
    }
    for(i = n - 1; i > 0; i--)
    {
        temp = a[0];
        a[0] = a[i];
        a[i] = temp;

        heapify(a, i, 0);
    }
}

void heapify(int a[], int n, int i)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    int temp;
    if(left < n && a[left] > a[largest])
    {
        largest = left;
    }
    if(right < n && a[right] > a[largest])

```



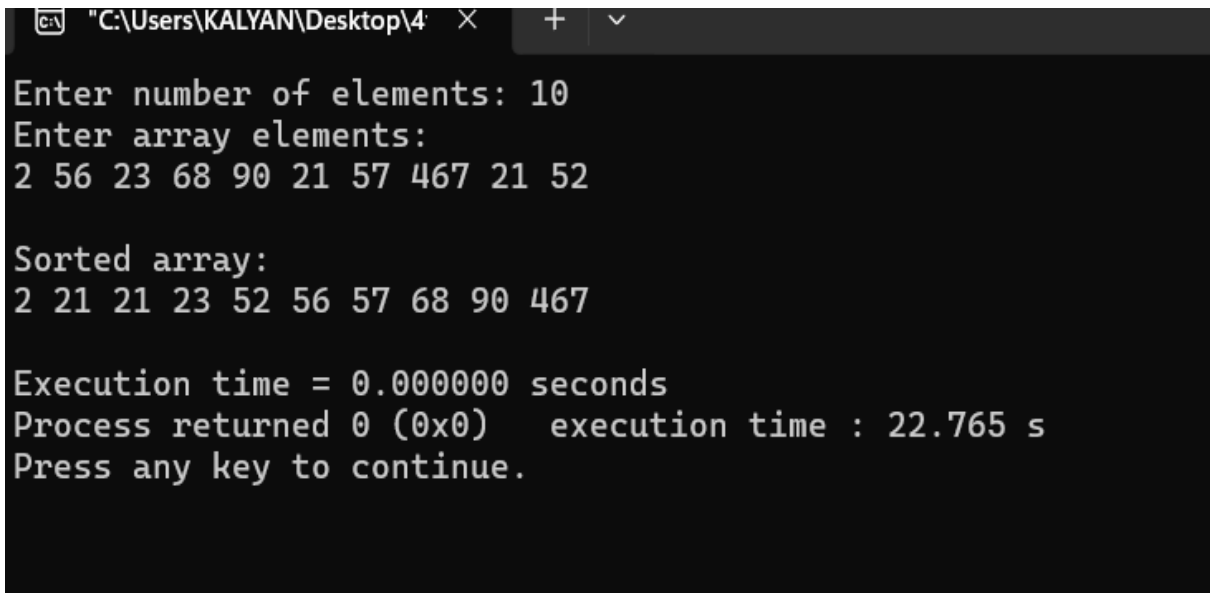
```

    {
        largest = right;
    }

    if(largest != i)
    {
        temp = a[i];
        a[i] = a[largest];
        a[largest] = temp;
        heapify(a, n, largest);
    }
}

```

Output:



```

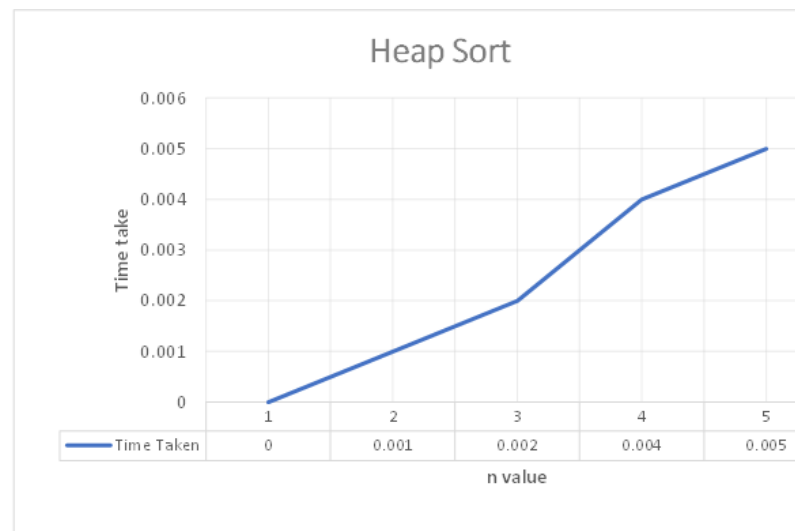
C:\Users\KALYAN\Desktop\4 >
Enter number of elements: 10
Enter array elements:
2 56 23 68 90 21 57 467 21 52

Sorted array:
2 21 21 23 52 56 57 68 90 467

Execution time = 0.000000 seconds
Process returned 0 (0x0)    execution time : 22.765 s
Press any key to continue.

```

Vertices (n)	Time Taken
1000	0
5000	0.001
10000	0.002
15000	0.004
25000	0.005



Lab Program 6:

Implement 0/1 Knapsack problem using dynamic programming.

```
#include <stdio.h>

void knapsack(int n, int m, int w[], int p[]) {

    int table[10][10];

    int x[10];

    for(int i = 0; i <= n; i++)

    {

        table[i][0]=0;

    }

    for(int j = 0; j<= m; j++)

    {

        table[0][j]=0;

    }

    for(int i = 1; i <= n; i++) {

        for(int j = 1; j <= m; j++) {

            if(w[i] > j)

                table[i][j] = table[i-1][j];

            else {

                if(p[i] + table[i-1][j - w[i]] > table[i-1][j])

                    table[i][j] = p[i] + table[i-1][j - w[i]];

                else

                    table[i][j] = table[i-1][j];

            }

        }

    }

    int i = n, j = m;

    while(i > 0 && j > 0) {

        if(table[i][j] != table[i-1][j])
```

```

        {
            x[i] = 1;
            j = j - w[i];
        }
        else{
            x[i]=0;
        }
        i--;
    }
    printf("\nMaximum Profit = %d\n", table[n][m]);
    printf("Selected items:");
    for(int i = 1; i <= n; i++)
        printf("%d ", x[i]);
    printf("\n");
}

int main() {
    int n, m, w[10], p[10];
    printf("Enter number of items: ");
    scanf("%d", &n);
    printf("Enter weights and profit of each items:\n");
    for(int i = 1; i <= n; i++)
        scanf("%d %d", &w[i], &p[i]);
    printf("Enter capacity: ");
    scanf("%d", &m);
    knapsack(n, m, w, p);
    return 0;
}

```

Output:

```
Enter number of items: 4
Enter weights and profit of each items:
1 10
2 15
4 5
3 20
Enter capacity: 5

Maximum Profit = 35
Selected items:0 1 0 1

Process returned 0 (0x0)    execution time : 34.525 s
Press any key to continue.
```

Lab Program 7:

Implement All Pair Shortest paths problem using Floyd's algorithm.

```
#include<stdio.h>

#define INF 9999

void floyds(int n)
{
    int d[n][n],i,j,k,a[n][n];

    printf("\nEnter the weight of adjacent matrix:");

    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
        {
            scanf("%d",&a[i][j]);
            if( i!=j && a[i][j]==0)
            {
                a[i][j]=INF;
            }
        }
    }

    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
            d[i][j]=a[i][j];
    }

    for(k=0;k<n;k++)
    {
        for(i=0;i<n;i++)
        {
            for(j=0;j<n;j++)
```

```

        {
            if(d[i][j]>d[i][k]+d[k][j])
                d[i][j]=d[i][k]+d[k][j];
        }
    }
}

printf("the shortest paths are:\n");
for(i=0;i<n;i++)
{
    for(j=0;j<n;j++)
    {
        printf("%d ",d[i][j]);
    }
    printf("\n");
}

}

int main()
{
    int n;
    printf("enter no.of vertices:");
    scanf("%d",&n);

    floyds(n);
    return 0;
}

```

Output:

```
enter no.of vertices:3
enter the weight of adjcent matrix:
0 2 7
2 0 4
7 4 0
the shortest paths are:
0 2 6
2 0 4
6 4 0

Process returned 0 (0x0)   execution time : 16.707 s
Press any key to continue.
```


Leet Code Program:

Given the head of a linked list, return *the list after sorting it in ascending order*.

</>Code

C  Auto

```
1  /**
2  * Definition for singly-linked list.
3  * struct ListNode {
4  *     int val;
5  *     struct ListNode *next;
6  * };
7  */
8  struct ListNode* sortList(struct ListNode* head) {
9
10     if(head == NULL)
11         return head;
12     struct ListNode *i, *j;
13     int temp;
14     for(i = head; i != NULL; i = i->next)
15     {
16         for(j = i->next; j != NULL; j = j->next)
17         {
18             if(i->val > j->val)
19             {
20                 temp = i->val;
21                 i->val = j->val;
22                 j->val = temp;
23             }
24         }
25     }
26     return head;
27 }
```

Output:

 Testcase |  Test Result

Accepted Runtime: 0 ms

 Case 1  Case 2  Case 3

Input

head =
[-1,5,3,4,0]

Output

[-1,0,3,4,5]

Expected

[-1,0,3,4,5]

✓ Testcase | >_ Test Result

Accepted Runtime: 0 ms

✓ Case 1 ✓ Case 2 ✓ Case 3

Input

```
head =  
[4,2,1,3]
```

Output

```
[1,2,3,4]
```

Expected

```
[1,2,3,4]
```

Accepted Runtime: 0 ms

✓ Case 1 ✓ Case 2 ✓ Case 3

Input

```
head =  
[]
```

Output

```
[]
```

Expected

```
[]
```

Lab Program 8:

a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

```
#include<stdio.h>

int prims(int num)
{
    int i,j,cost[35][35],a,b,v,u,visited[35],ne=0,mincost=0;
    printf("enter weights:");
    for(i=0;i<num;i++)
    {
        for(j=0;j<num;j++)
        {
            scanf("%d",&cost[i][j]);
            if(cost[i][j]==0)
                cost[i][j]=999;
        }
    }
    visited[0]=1;
    while(ne<num-1)
    {
        int min=999;
        for(i=0;i<num;i++)
        {
            for(j=0;j<num;j++)
            {
                if(cost[i][j]<min)
                {
                    if(visited[i]!=0)
                    {
                        min=cost[i][j];

```

```

        a=u=i;
        b=v=j;
    }
}
}

printf("\nn%d->%d=%d",a,b,min);
mincost+=min;
visited[b]=1;
ne++;
cost[a][b]=cost[b][a]=999;
}
printf("\nmincost:%d",mincost);
}
int main()
{
    int num;
    printf("ENTER no.of nodes:");
    scanf("%d",&num);
    prims(num);
}

```

Output:

```

ENTER no.of nodes:4
enter weights:
0 2 0 6
2 0 3 8
0 3 0 5
6 8 5 0

0->1=2
1->2=3
2->3=5
mincost:10
Process returned 0 (0x0)   execution time : 97.843 s
Press any key to continue.

```

b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

```
#include<stdio.h>

int parent[35];

int find(int i)
{
    while(parent[i])
        i=parent[i];
    return i;
}

int uni(int i,int j)
{
    if(i!=j)
    {
        parent[j]=i;
        return 1;
    }
    return 0;
}

int main()
{
    int n,e,cost[35][35],u,v,a,b,i,j,ne=0;
    int min,mincost=0;
    printf("enter no.of nodes:");
    scanf("%d",&n);
    printf("\nreading weight of graph:");
    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
        {
```

```

scanf("%d",&cost[i][j]);

if(cost[i][j]==0)
    cost[i][j]=999;
}
}
for(i=0;i<n;i++)
{
    parent[i]=0;
}
while(ne<n-1)
{
    min=999;
    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
        {
            if(cost[i][j]<min)
            {
                min=cost[i][j];
                a=u=i;
                b=v=j;
            }
        }
    }
    u=find(u);
    v=find(v);
    if(uni(u,v))
    {
        printf("\n%d->%d=%d",a,b,min);
    }
}

```

```

        mincost+=min;

        ne++;
    }

    cost[a][b]=cost[b][a]=999;
}

printf("\nmin cost is:%d",mincost);
}

```

Output:

```

enter no.of nodes:4

reading weight of graph:
0 2 0 6
2 0 3 8
0 3 0 5
6 8 5 0

0->1=2
1->2=3
2->3=5
min cost is:10
Process returned 0 (0x0)   execution time : 43.248 s
Press any key to continue.
|

```

Lab Program 9:

Implement Fractional Knapsack using Greedy technique.

```
#include<stdio.h>

int main()
{
    int c, n, i, j;
    int w[35], p[35];
    float r[35], tp = 0, temp;
    printf("Enter the no. of items: ");
    scanf("%d", &n);
    printf("Enter the capacity: ");
    scanf("%d", &c);
    printf("Enter weights of items:\n");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &w[i]);
    }
    printf("Enter profit of items:\n");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &p[i]);
    }
    for(i = 0; i < n; i++)
    {
        r[i] = (float)p[i] / w[i];
    }
    for(i = 0; i < n; i++)
    {
        for(j = i + 1; j < n; j++)
```



```

    {
        if(r[i] < r[j])
        {
            temp = r[i];
            r[i] = r[j];
            r[j] = temp;
            temp = w[i];
            w[i] = w[j];
            w[j] = temp;
            temp = p[i];
            p[i] = p[j];
            p[j] = temp;
        }
    }
}

for(i = 0; i < n; i++)
{
    if(w[i] <= c)
    {
        tp = tp + p[i];
        c = c - w[i];
    }
    else
    {
        tp = tp + (r[i] * c);
        break;
    }
}

printf("\nMaximum profit = %.2f", tp);

```

```
return 0;
```

```
}
```

Output:

```
Enter the no. of items: 4
Enter the capacity: 25
Enter weights of items:
5 10 15 20
Enter profit of items:
30 40 45 77

Maximum profit = 108.50
Process returned 0 (0x0)   execution time : 24.353 s
Press any key to continue.
|
```

Lab Program 10:

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

```
#include<stdio.h>

void dijkstras(int c[30][30],int n,int s,int d[30])
{
    int i,u,min,j,v[15];
    for(i=0;i<n;i++)
    {
        d[i]=c[s][i];
        v[i]=0;
    }
    v[s]=1;
    for(i=0;i<n;i++)
    {
        min=9999;
        for(j=0;j<n;j++)
        {
            if(v[j]==0&& d[j]<min)
            {
                min=d[j];
                u=j;
            }
        }
        v[u]=1;
        for(j=0;j<n;j++)
        {
            if(v[j]==0&& d[u]+c[u][j]<d[j])
                d[j]=d[u]+c[u][j];
        }
    }
}
```

```

    }

    printf("\nthe shortest distance are:");

    for(i=0;i<n;i++)
    {
        printf("\n%d->%d=%d",s,i,d[i]);
    }
}

```

```

int main()
{
    int n,s;

    printf("enter no.of nodes:");

    scanf("%d",&n);

    int c[30][30],d[30];

    printf("\nenter cost of edges:");

    for(int i=0;i<n;i++)
    {
        for(int j=0;j<n;j++)
        {
            scanf("%d",&c[i][j]);

            if(c[i][j]==0)
                c[i][j]=9999;
        }
    }

    printf("\nenter the start vertex:");

    scanf("%d",&s);

    dijkstras(c,n,s,d);
}

```

Output:

```
C:\Users\KALYAN\Desktop\4 x + v
enter no.of nodes:4
enter cost of edges:
0 2 0 1
2 0 3 2
0 3 0 4
1 2 4 0

enter the start vertex:0

the shortest distance are:
0->0=9999
0->1=2
0->2=5
0->3=1
Process returned 0 (0x0)    execution time : 20.668 s
Press any key to continue.
|
```

Lab Program 11:

Implement the “N-Queens Problem” using Backtracking.

```
#include<stdio.h>

#include<stdlib.h>

#define max 50

int can_place(int c[],int r)
{
    int i;
    for(i=0;i<r;i++)
    {
        if(c[i]==c[r] || (abs(c[i]-c[r])==abs(i-r)))
        {
            return 0;
        }
    }
    return 1;
}

void display(int c[],int n)
{
    int i,j;
    char cb[10][10];
    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
        {
            cb[i][j]='-';
        }
    }
    for(i=0;i<n;i++)
```

```

    {
        cb[i][c[i]]='Q';
    }
    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
        {
            printf("%c",cb[i][j]);
        }
        printf("\n");
    }
}

void n_queens(int n)
{
    int r;
    int c[max];
    c[0]=-1;
    r=0;
    while(r>=0)
    {
        c[r]++;
        while(c[r]<n&&!can_place(c,r))
            c[r]++;
        if(c[r]<n)
        {
            if(r==n-1)
            {
                display(c,n);
                printf("\n\n");
            }
        }
    }
}

```

```

    }
    else{
        r++;
        c[r]=-1;
    }
}
else{
    r--;
}
}
}
int main()
{
    int n;
    printf("enter the no.of queens:");
    scanf("%d",&n);
    n_queens(n);
}

```

Output:

```

C:\Users\KALYAN\Desktop\4  X  +  v
enter the no.of queens:4
-Q--
---Q
Q---
--Q-

--Q-
Q---
---Q
-Q--

Process returned 0 (0x0)   execution time : 1.503 s
Press any key to continue.
|

```